



DIPARTIMENTO DI INGEGNERIA INFORMATICA, MODELLISTICA,
ELETTRONICA E SISTEMISTICA

Corso di Laurea Magistrale in Ingegneria Informatica

Machine Learning

Continual Adaptation in Industrial Anomaly Detection

Rocco Pio Vardaro 264097

ANNO ACCADEMICO 2025-2026

Indice

Indice	i
1 Introduzione	1
1.1 Contesto	1
1.2 Definizione del Problema	2
1.3 Obiettivi del Progetto	2
2 Analisi del Paper	5
2.1 Il ponte tra AD e CL: Data Augmentation	5
2.2 Il Metodo DNE (Distribution of Normal Embeddings)	6
2.2.1 Fase di Addestramento	7
2.2.2 Fase di Inferenza	9
2.3 Metriche Utilizzate	10
3 Sviluppo Architetture e Soluzioni Proposte	12
3.1 Architetture di Estrazione delle Feature	12
3.1.1 Vision Transformer (ViT-16)	13
3.1.2 Convolutional Neural Network (CNN)	13
3.2 Metodi di CAD implementati	14
3.2.1 DNE Standard	14
3.2.2 DNE_EWC	15
3.2.3 DNE_Replay_EWC	16
4 Dataset e Scenari di Valutazione	18
4.1 Dataset Paper: MVTec AD	18

4.2	Dataset VisA	18
4.3	Definizione dei Task Sequenziali	19
5	Risultati Sperimentali	21
5.1	Risultati ottenuti su Dataset MVTec AD	21
5.1.1	Analisi del metodo base: DNE	21
5.1.2	Analisi del metodo DNE_EWC	24
5.1.3	Analisi del metodo DNE_replay_EWC	27
5.1.4	Iperparametri di Addestramento	31
5.2	Risultati ottenuti su Dataset VisA	32
5.2.1	Analisi del metodo base: DNE	33
5.2.2	Analisi del metodo DNE_EWC	36
5.2.3	Analisi del metodo DNE_replay_EWC	39
5.2.4	Iperparametri di Addestramento	42
6	Conclusioni e Sviluppi Futuri	44
6.1	Conclusioni	44
6.2	Sviluppi Futuri	45

Capitolo 1

Introduzione

1.1 Contesto

L'ispezione visiva e il controllo qualità rappresentano fasi cruciali all'interno dei moderni scenari di produzione industriale. Storicamente, l'identificazione di difetti sui prodotti è stata affidata all'ispezione visiva umana, un processo intrinsecamente lento, costoso e soggetto a errori derivanti dall'affaticamento o dalla soggettività dell'operatore. Con l'avvento del Deep Learning, i sistemi di visione artificiale hanno rivoluzionato questo settore, offrendo soluzioni automatizzate e ad alta precisione.

Tuttavia, l'applicazione di algoritmi di classificazione supervisionata classici risulta spesso impraticabile: in un contesto industriale reale, i campioni difettosi (anomalie) sono rari, estremamente eterogenei e difficili da raccogliere in quantità sufficienti per addestrare un modello robusto. Per superare questo ostacolo, ha assunto un'importanza centrale l'Anomaly Detection (AD) non supervisionata, un paradigma in cui il modello viene addestrato esclusivamente su campioni "normali" (privi di difetti). Durante la fase di inferenza, il sistema impara a riconoscere come anomalia qualsiasi deviazione significativa rispetto alla distribuzione dei dati normali appresa.

Nel mondo reale della produzione industriale, questa assunzione è raramente verificata. Le linee di produzione sono ambienti dinamici in continua evoluzione: nuovi componenti o prodotti vengono frequentemente introdotti, le

condizioni di illuminazione possono variare nel tempo e i sensori o le telecamere possono essere aggiornati.

1.2 Definizione del Problema

La transizione da un paradigma di apprendimento statico (offline) a uno continuo (online o sequenziale) introduce una delle sfide aperte più note nel campo delle reti neurali artificiali: il **Catastrophic Forgetting**.

Quando una rete neurale viene addestrata sequenzialmente su una serie di task, i pesi della rete vengono ottimizzati per minimizzare la funzione di perdita relativa all'ultimo task presentato. In assenza di meccanismi di preservazione della memoria, questo aggiornamento dei pesi altera irrimediabilmente le rappresentazioni interne precedentemente apprese. Nel contesto dell'Anomaly Detection (AD), se un modello viene prima addestrato a riconoscere la normalità di una "vite" (Task 1) e successivamente viene aggiornato per ispezionare una "scheda elettronica" (Task 2), il suo spazio latente delle feature si modificherà per accogliere i pattern della scheda. Di conseguenza, il modello perderà la capacità di ricostruire o mappare correttamente le feature della vite, iniziando a classificarne i campioni normali come anomalie.

Nasce così la formulazione del task di **Continual Anomaly Detection (CAD)**. In questo scenario, il problema è definito come segue: dato un flusso sequenziale di dataset $\mathcal{D} = D_1, D_2, \dots, D_T$, dove ogni D_t rappresenta l'insieme delle immagini normali per un nuovo prodotto (task t), l'obiettivo è addestrare un singolo modello unificato che impari a identificare le anomalie per il task corrente t , mantenendo al contempo elevate prestazioni di rilevamento sui task precedenti $1, \dots, t-1$.

1.3 Obiettivi del Progetto

L'obiettivo principale di questo progetto è quello di espandere e migliorare le fondamenta teoriche e pratiche poste dal paper "*Towards Continual*

Adaptation in Industrial Anomaly Detection". Mentre lo studio originale propone la tecnica DNE associata a un'architettura ViT-B/16 come soluzione base per il Continual Anomaly Detection, questo lavoro introduce diverse innovazioni metodologiche e architetturali volte a migliorare l'efficienza del modello in termini di risorse computazionali (utilizzando reti neurali più efficienti) ed incrementare la robustezza dei modelli contro il catastrophic forgetting.

Nello specifico, il progetto si articola sui seguenti obiettivi primari:

- **Riproduzione e Validazione della Baseline:** Implementazione fedele del metodo originale DNE abbinato all'architettura Vision Transformer (ViT-16). Questo step è fondamentale per stabilire un punto di riferimento solido (baseline) con cui confrontare le successive estensioni.
- **Estensione Architetturale:** Sostituzione e confronto del ViT-16 con reti neurali di estrazione delle feature computazionalmente più efficienti. Il framework originale è stato esteso introducendo le architetture ConvNeXt ed EfficientNet.
- **Integrazione e Sviluppo di Nuove Strategie basate su DNE:** Al fine di mitigare ulteriormente il degrado delle prestazioni sui vecchi task, la metodologia DNE è stata potenziata sviluppando due nuove varianti algoritmiche:
 - **DNE EWC:** Integrazione dell'algoritmo Elastic Weight Consolidation (EWC)[1]. Questo approccio introduce un vincolo di regolarizzazione durante l'aggiornamento della rete, calcolando la matrice di Informazione di Fisher per penalizzare le modifiche a quei pesi considerati cruciali per il rilevamento di anomalie sui prodotti passati.
 - **DNE Replay EWC:** Sviluppo di un approccio metodologico ibrido e innovativo. Questa variante risolve il problema dell'obsolescenza delle feature latenti (stale embeddings) combinando l'azione di regolarizzazione di EWC con un meccanismo di Image Replay.

- **Validazione Sperimentale su Nuovi Dataset Complessi:** Oltre alla validazione sul dataset utilizzato nel paper, il progetto compie uno step aggiuntivo introducendo il test sul dataset **VisA** (Visual Anomaly Detection).

Capitolo 2

Analisi del Paper

Il divario tra Anomaly Detection (AD) e Continual Learning (CL) ha storicamente ostacolato lo sviluppo di sistemi di controllo qualità capaci di evolvere nel tempo. Gran parte dei metodi di CL esistenti si concentra su un paradigma supervisionato, inadatto all'AD dove le anomalie sono incognite. D'altro canto, i metodi di AD si concentrano unicamente sul dataset corrente, subendo un drastico catastrophic forgetting quando esposti a nuovi dati non stazionari.

Per colmare questa lacuna, il progetto prende in esame e fa da estensione al framework proposto nel paper "*Towards Continual Adaptation in Industrial Anomaly Detection*" (Wujin Li et al., 2022). Il paper introduce una strategia unificata che sfrutta la data augmentation come ponte tra i due mondi e propone il metodo DNE (Distribution of Normal Embeddings).

Di seguito viene analizzata in maniera approfondita la metodologia alla base di DNE.

2.1 Il ponte tra AD e CL: Data Augmentation

L'intuizione fondamentale che permette di applicare tecniche di Continual Learning all'Anomaly Detection è la conversione del problema da non supervisionato a parzialmente supervisionato (o self-supervised). DNE adot-

ta strategie di data augmentation avanzate per generare campioni anomali a partire dalle immagini normali a disposizione.

Il modello, composto da un estrattore di feature f_e (backbone) e da un classificatore h (head), viene addestrato con una funzione di loss basata sulla Cross-Entropy (CE), che penalizza l'errata classificazione tra un'immagine normale x (etichetta 0) e un'immagine con anomalia sintetica $\tilde{x}$ (etichetta 1):

$$L_{tr} = CE[h(f_e(x)), 0] + CE[h(f_e(\tilde{x})), 1]$$

Questa rappresentazione pseudo-supervisionata è essenziale poiché costringe lo spazio latente del backbone a separare le feature normali da quelle anomale, creando una base solida su cui innestare la preservazione statistica di DNE.

2.2 Il Metodo DNE (Distribution of Normal Embeddings)

L'approccio **DNE** si fonda sull'idea che, invece di conservare direttamente le immagini grezze dei task precedenti — come avviene nei metodi di rehearsal tradizionali (ad esempio ER o DER), con un elevato costo in termini di memoria e potenziali criticità legate alla privacy — sia più efficiente **memorizzare soltanto le statistiche delle rappresentazioni interne dei dati**.

In particolare, DNE assume che le feature estratte da una rete neurale pre-addestrata possano essere modellate tramite una **distribuzione gaussiana**. Sulla base di questa ipotesi, il modello non salva i dati originali, ma utilizza la rete per estrarre le caratteristiche rilevanti delle immagini “normali” (ossia prive di anomalie) e le riassume sotto forma di distribuzioni statistiche.

Queste distribuzioni vengono poi compresse memorizzando unicamente i loro parametri fondamentali, cioè **media** e **covarianza**, riducendo in modo significativo il fabbisogno di memoria pur preservando l'informazione essenziale acquisita durante l'apprendimento.

In fase di valutazione, il sistema ricostruisce implicitamente le distribuzioni apprese a partire da tali statistiche e confronta i nuovi campioni con i modelli precedenti. In sintesi, DNE rappresenta un ponte tra anomaly detection e continual learning, permettendo di mantenere conoscenza nel tempo in modo efficiente, scalabile e rispettoso dei vincoli di memoria e privacy.

2.2.1 Fase di Addestramento

La fase di **addestramento** si sviluppa per ogni nuovo task sequenziale e si articola nei seguenti passaggi:

- **Estrazione delle Feature e Calcolo della Loss:** Per ogni mini-batch di immagini, la rete pre-addestrata (composta dal feature extractor f_e e dalla testa di classificazione h) processa le immagini normali x e le loro versioni aumentate $\bar{x}$ (usate come anomalie simulate). La funzione di loss utilizzata per l'addestramento è la Cross-Entropy (CE):

$$L_{tr} = CE[h(f_e(x); \theta), 0] + CE[h(f_e(\bar{x}); \theta), 1]$$

Per evitare alterazioni drastiche delle distribuzioni, i parametri θ della testa h vengono aggiornati solo durante il primo task e congelati per tutti i task successivi.

- **Buffer Temporaneo:** Durante l'intera epoca di addestramento per il task corrente τ , tutti gli embedding Z estratti dal backbone vengono salvati in una lista temporanea:

$$Z_{epoch} = \{Z_1, Z_2, \dots, Z_N\}$$

dove $N = \lceil n_\tau / B \rceil$, con n_τ pari al numero di campioni totali del task e B la dimensione del batch.

- **Stima Statistica e Covarianza Empirica:** Alla fine dell'epoca, assumendo una distribuzione Gaussiana a priori, il sistema calcola la media empirica $\bar{z}$ e la matrice di covarianza empirica $\hat{\Sigma}$ sugli embedding salvati:

$$\hat{\Sigma} = \frac{1}{n-1} \sum_{i=1}^n (z_i - \bar{z})(z_i - \bar{z})^\top$$

- **Trasformazione "Shrinkage":** Dato che i dataset industriali spesso hanno pochi campioni rispetto all'alta dimensionalità D delle feature estratte, la matrice di covarianza empirica rischia di non essere invertibile per via di problemi numerici. Per correggere questo difetto, viene applicata una regolarizzazione convessa detta *shrinkage*:

$$\hat{\Sigma}_{shrink} = (1 - \alpha)\hat{\Sigma} + \alpha \frac{tr(\hat{\Sigma})}{D} I_d$$

dove $\alpha \in [0, 1]$ è il peso dello shrinkage e I_d è la matrice identità.

- **Aggiornamento della Memoria:** Il buffer temporaneo delle immagini viene distrutto. Solo le **statistiche finali essenziali del task** vengono salvate nella memoria a lungo termine M , registrando specificamente una tupla composta da: *numero di campioni, media e matrice di covarianza stabilizzata* $([n_\tau, \mu, \hat{\Sigma}_{shrink}])$

Algorithm 1: CAD Training Loop

```

1 Input: pre-trained backbone  $f_e$  and head  $h$ , episodic memory  $M$ ,
   number of epochs  $E$ 
2 Initialize:  $\mathcal{M} \leftarrow \phi$ 
3 for  $t = 1, \dots, T$  do
4   Construct dataset  $\mathcal{D}_{train}^t \leftarrow \{x_i^t\}$ 
5   for  $e = 1, \dots, E$  do
6     Receive a mini-batch  $\{X_k\}_{k=1}^N$  from  $\mathcal{D}_{train}^t$  and construct
       a temporary list  $Z_{epoch}$ 
7     for all  $k \in \{1, \dots, N\}$  do
8        $Z_k = f_e(X_k)$ 
9        $Z_{epoch}.append(Z_k)$ 
10       $Y_k = h(Z_k)$ 
11      Update the backbone  $f_e$ 
12      Update the head  $h$  if  $t == 0$  else fix head  $h$ 
13    end
14     $nums, \mu, \Sigma \leftarrow Z_{epoch}$  //Based on the features stored in
       $Z_{epoch}$ , calculating its numbers, mean  $\mu$  and covariance
       $\Sigma$ .
15     $M[-1] = [nums, \mu, \Sigma]$ 
16  end
17   $M.append([nums, \mu, \Sigma])$ 
18 end
19 Return: pre-trained backbone  $f_e$ , head  $h$ , episodic memory  $M$ 

```

Figura 2.1: Algoritmo di Training DNE

2.2.2 Fase di Inferenza

Durante la fase di test, il modello deve dedurre se una nuova immagine è anomala senza conoscere a priori da quale task provenga. Il processo si articola nei seguenti passaggi:

- **Ricostruzione delle Distribuzioni Passate:** Per ogni task t precedentemente archiviato nella memoria episodica M , il sistema recupera le statistiche salvate: il numero di campioni ($nums_t$), la media (μ_t) e la covarianza (Σ_t). Utilizzando questi parametri, ricostruisce la distribuzione Gaussiana specifica per quel task.
- **Campionamento e Unione:** Da ciascuna di queste distribuzioni ricostruite, il modello campiona un numero di punti virtuali pari a $nums_t$. Tutti i punti campionati dai vari task vengono uniti in un'unica lista temporanea (chiamata Z_{tasks}). Questo insieme globale funge da nuova distribuzione Gaussiana complessiva che rappresenta tutte le caratteristiche apprese nel passato.
- **Calcolo delle Statistiche Globali:** A partire da questo insieme combinato di punti, vengono calcolate la nuova media globale $\bar{z}$ e la nuova matrice di covarianza globale $\hat{\Sigma}_{shrunk}$.
- **Estrazione delle Feature di Test:** Data un'immagine di test x , il sistema utilizza il backbone pre-addestrato f_e per estrarne il relativo vettore di embedding z :

$$z = f_e(x)$$

- **Calcolo del Punteggio di Anomalia:** Infine, il modello calcola la Distanza di Mahalanobis (Md) tra l'embedding di test z e la distribuzione globale appena costruita. Questa distanza, che funge da punteggio di anomalia definitivo.

$$Md(z) = \sqrt{(z - \bar{z})^\top \hat{\Sigma}_{shrunk}^{-1} (z - \bar{z})}$$

Algorithm 2: CAD Inference Loop

```

1 Input: test dataset  $D_{test}$ , backbone  $f_e$ , episodic memory  $M$ 
2 for  $t$  in  $len(M)$  do
3    $nums_t, \mu_t, \Sigma_t = M[t]$ 
4    $\varphi_t(\mu, \Sigma; z) \leftarrow \mu_t, \Sigma_t$  // The Gaussian distribution of task  $t$  is
      reconstructed based on the mean  $\mu_t$  and covariance  $\Sigma_t$  .
5    $\hat{Z}_t \leftarrow$  sample  $nums_t$  points from  $\varphi_t(\mu, \Sigma_t)$ 
6   construct a temporary list  $Z_{tasks}$ 
7    $Z_{tasks}.append(\hat{Z}_t)$ 
8 end
9  $nums, \mu, \Sigma \leftarrow Z_{tasks}$ 
10 Given a test image  $x$ 
11  $z = f_e(x)$ 
12  $Md \leftarrow \sqrt{(z - \mu)^\top \hat{\Sigma}_{shrunk}^{-1} (z - \mu)}$ .
13 Return: anomaly score  $Md$ 

```

Figura 2.2: Algoritmo di Inference DNE

2.3 Metriche Utilizzate

Per quantificare in modo rigoroso le capacità di apprendimento e di ritenzione delle informazioni, vengono utilizzate due metriche fondamentali: l'*Average AUC* (Area Under the Curve) e la *Forgetting Measure*.

Per definire matematicamente queste metriche, introduciamo prima la seguente notazione formale:

- N : rappresenta il numero totale di task presenti nell'intera sequenza di apprendimento continuo.
- i : è l'indice del task su cui il modello ha appena terminato la fase di addestramento (con $1 \leq i \leq N$).
- j : è l'indice del task su cui il modello viene testato e valutato in un dato momento (con $j \leq i$, indicando che valutiamo il modello sui task già visti).
- $A_{i,j}$: rappresenta il valore della prestazione (nel nostro caso misurata tramite la metrica AUC) che il modello ottiene sul set di test del

task j , calcolata esattamente dopo che il modello è stato addestrato in sequenza fino al task i .

1. **Average AUC:** stima la performance complessiva del modello al termine dell'intero processo di apprendimento continuo. È definita come la media aritmetica delle prestazioni AUC ottenute su tutti gli N task, valutate dopo che il modello ha completato l'addestramento sull'ultimo task. Questa metrica indica quanto il modello è accurato globalmente al termine del ciclo di training:

$$AUC = \frac{1}{N} \sum_{j=1}^N A_{N,j}$$

2. **Forgetting Measure:** quantifica il fenomeno del *catastrophic forgetting*, ovvero la tendenza di una rete neurale a dimenticare le informazioni apprese sui task precedenti quando viene esposta a nuovi dati. Il forgetting relativo a un singolo task j è calcolato come la differenza tra la massima prestazione ottenuta su quel task nel passato e la prestazione attuale:

$$f_{N,j} = \left(\max_{k \in \{1, \dots, N-1\}} A_{k,j} \right) - A_{N,j}$$

La Forgetting Measure media complessiva (FM), al termine degli N task, si ottiene calcolando la media dei valori di forgetting per tutti i task precedenti:

$$FM = \frac{1}{N-1} \sum_{j=1}^{N-1} f_{N,j}$$

Un valore di FM basso, associato ad un valore di AUC alto, indica che il modello riesce a mantenere intatta o a migliorare la conoscenza acquisita nei task precedenti, resistendo in modo efficace al catastrophic forgetting.

Capitolo 3

Sviluppo Architetture e Soluzioni Proposte

3.1 Architetture di Estrazione delle Feature

Nell’approccio DNE, la qualità della rappresentazione delle feature estratte dai dati in ingresso gioca un ruolo fondamentale. Un **estrattore di feature (backbone)** robusto deve essere in grado di mappare immagini normali e anomale in uno spazio latente ben separabile, pur mantenendo una stabilità rappresentazionale man mano che il modello apprende nuovi task. In questo lavoro, le prestazioni dei metodi utilizzati sono state valutate e confrontate utilizzando diverse architetture, esplorando sia paradigmi basati sull’attenzione che approcci convoluzionali. Le backbone utilizzate sono:

- Vision Transformer (ViT-16)
- EfficientNet
- ConvNeXt

3.1.1 Vision Transformer (ViT-16)

ViT elabora l'immagine suddividendola in una griglia di patch di dimensione fissa (es. 16x16 pixel), trattandole come una sequenza di token. Attraverso l'utilizzo del meccanismo di **Self-Attention**, ViT è in grado di catturare le dipendenze globali e il contesto spaziale a lungo raggio fin dai primissimi layer della rete.

3.1.2 Convolutional Neural Network (CNN)

I modelli come ViT-16 tendono a richiedere elevate quantità di dati per generalizzare efficacemente e presentano un onere computazionale non trascurabile. Pertanto, il progetto è stato esteso integrando due architetture basate su Convolutional Neural Networks: **EfficientNet** e **ConvNeXt**.

EfficientNet

EfficientNet è una famiglia di reti neurali convoluzionali sviluppata da Google che mira a ottenere il miglior compromesso tra **accuratezza** ed **efficienza computazionale**. La sua principale innovazione è il **Compound Scaling**, un metodo che ridimensiona in modo bilanciato profondità, larghezza della rete e risoluzione delle immagini.

La versione base, **EfficientNet-B0**, è caratterizzata da un numero ridotto di parametri e un'elevata efficienza.

Nel progetto è stata utilizzata una versione di **EfficientNet-B0** [3], implementato nel file `efficientnet.py`, con pesi pre-addestrati su ImageNet. Il modello originale è stato modificato rimuovendo il classificatore finale e utilizzando la rete come feature extractor, ottenendo un vettore di caratteristiche da 1280 dimensioni. È stata aggiunta una nuova testa di classificazione composta da livelli fully connected.

ConvNeXt

ConvNeXt è una famiglia di reti neurali convoluzionali che modernizza le architetture convoluzionali tradizionali integrando alcune tecniche ispirate ai Vision Transformer. Basata sull'evoluzione di ResNet, introduce miglioramenti come **depthwise convolution**, **kernel più grandi (7×7)**, **Layer Normalization** e attivazione **GELU**, ottenendo prestazioni elevate mantenendo un'architettura completamente convoluzionale, efficiente e scalabile.

Nel progetto è stata utilizzata **ConvNeXt-Tiny [2]**, implementata nel file `convnext.py`, pre-addestrata su ImageNet come **feature extractor** per l'analisi dello spazio delle caratteristiche. Il classificatore originale è stato rimosso e le feature estratte vengono trasformate in un vettore di dimensione 768. È stato aggiunto un proiettore non lineare che riduce le feature in uno spazio embedding più compatto (256 dimensioni), utilizzando Linear Layer, LayerNorm e GELU.

3.2 Medoti di CAD implementati

Al fine di valutare e migliorare le capacità di adattamento continuo dei modelli, in questo progetto sono stati implementati, testati e confrontati tre distinti approcci metodologici per mitigare il catastrophic forgetting. Partendo dalla baseline (DNE), la complessità e la robustezza dei metodi sono state incrementate fino a proporre una soluzione ibrida originale estremamente robusta.

3.2.1 DNE Standard

Il **metodo DNE** costituisce la baseline di riferimento del progetto ed è implementato secondo quanto proposto nel paper originale. L'approccio modella la distribuzione di normalità memorizzando, al termine di cia-

scun task, esclusivamente le statistiche degli embedding estratti, ossia la media μ e la matrice di covarianza Σ .

Il principale limite di questo metodo risiede nel fatto che, nonostante l'adozione della Fixed Head, i pesi della backbone continuano a subire lievi aggiornamenti nel corso dell'apprendimento. Di conseguenza, le statistiche salvate nei task precedenti (μ, Σ) non risultano più pienamente rappresentative del nuovo spazio delle feature appreso dalla backbone corrente, dando origine al fenomeno degli **stale embeddings**¹.

3.2.2 DNE_EWC

Per frenare attivamente la deriva dei pesi del backbone durante l'apprendimento di task sequenziali, è stata sviluppata l'**estensione DNE_EWC**, implementato nel file `dne_ewc.py`. Questo metodo integra la logica dell'Elastic Weight Consolidation (EWC) [1].

L'obiettivo è **proteggere in modo selettivo i parametri della rete neurali** che sono risultati fondamentali per riconoscere le anomalie nei task precedenti.

L'implementazione algoritmica di DNE_EWC opera come segue:

- **Calcolo Cumulativo dell'Informazione di Fisher:** Al termine di ogni task k , il modello esegue un loop sui dati di addestramento calcolando i gradienti della loss. Il calcolo della matrice di Fisher in EWC serve a misurare l'"importanza" di ciascun peso della rete per il task appena concluso. Matematicamente, si utilizza un'approssimazione chiamata **Empirical Fisher**, limitandosi ai soli elementi sulla diagonale (per motivi di efficienza e memoria).

¹Gli stale embeddings nella backbone sono rappresentazioni vettoriali generate da una versione precedente della backbone, che non riflettono più correttamente le caratteristiche apprese dalla versione corrente del modello.

$$F_i = \frac{1}{N} \sum_{k=1}^N \left(\frac{\partial \mathcal{L}(x_k, y_k; \theta)}{\partial \theta_i} \right)^2$$

Dove:

- * F_i è il valore sulla diagonale della matrice di Fisher per il parametro θ_i .
 - * N è il numero totale di campioni nel dataset.
 - * $\mathcal{L}$ è la funzione di Loss (nel tuo caso la Cross-Entropy).
 - * $\frac{\partial \mathcal{L}}{\partial \theta_i}$ è il gradiente (derivata) della loss rispetto al parametro θ_i per il campione k .
- **Archiviazione dello Stato Ottimale:** Insieme alla matrice di Fisher, vengono salvati in memoria i pesi ottimali raggiunti dalla rete alla fine del task (θ_k^*). Ogni task passato ha il suo dizionario dedicato di parametri ottimali e matrici di Fisher.
 - **Loss di Regolarizzazione (EWC Penalty):** Durante il training sui task successivi ($t \geq 1$), alla Cross-Entropy Loss di base viene sommato un termine di regolarizzazione calcolato in tempo reale su tutti i task precedentemente affrontati:

$$L_{totale} = L_B(\theta) + \frac{\lambda}{2} \sum_{k=0}^{t-1} \sum_i F_{k,i} (\theta_i - \theta_{k,i}^*)^2$$

dove λ modula l'intensità della penalità. Questa somma garantisce che l'apprendimento del nuovo prodotto eviti drasticamente di alterare i pesi per le vecchie distribuzioni.

3.2.3 DNE_Replay_EWC

Per risolvere in via definitiva l'obsolescenza delle feature statistiche passate, in questo lavoro è stato sviluppato l'approccio ibrido DNE_Replay_EWC, implementato nel file `dne_replay_ewc.py`. Questo metodo massimizza l'affidabilità della fase di inferenza unendo tre approcci distinti: **metodo DNE, EWC, Image Replay**.

Le innovazioni fondamentali si concentrano sulla fase di `end_task` e sulla preparazione della densità di inferenza:

- **Image Replay Buffer:** Al termine di ogni task, oltre a calcolare e salvare la Matrice di Fisher come in DNE_EWC, il metodo seleziona in modo casuale un sottoinsieme molto ristretto, `real_embed_ratio` (es. 5%), di immagini del prodotto appena appreso. Queste immagini vengono salvate senza subire alcuna data augmentation per garantire che rappresentino la normalità pura.
- **Ricalcolo degli Embedding:** Durante la fase in cui il modello ricostruisce le distribuzioni storiche per validare le anomalie, non si limita a usare vettori passati. Prende il buffer di immagini originali salvate e le fa processare dalla backbone corrente. In questo modo, genera dei **Real Embeddings** che sono perfettamente aggiornati all'ultimissimo stato della rete, eliminando in via definitiva il problema degli stale embedding.
- **Stima Ibrida della Densità Gaussiana:** Poiché salvare troppe immagini violerebbe i limiti di memoria del Continual Learning, i pochi "*Real Embeddings*" estratti non bastano a formare una distribuzione densa. Il metodo li combina dinamicamente con:
 - * **Embeddings Sintetici:** Generati campionando dalla gaussiana multivariata delle vecchie medie (μ) e covarianze (Σ) salvate nel passato.
 - * **Rumore Casuale Multivariato:** Aggiunto in base a una percentuale prestabilita (`noise_ratio`) per allargare le tolleranze dei margini di normalità.

Capitolo 4

Dataset e Scenari di Valutazione

Per misurare in modo oggettivo le prestazioni delle architetture (ViT, ConvNeXt, EfficientNet) e dei metodi di Continual Learning (DNE, DNE_EWC, DNE_Replay_EWC), è fondamentale testarli su benchmark industriali realistici. Oltre al dataset utilizzato nel paper, in questo progetto è stato introdotto uno scenario sperimentale su un dataset ad alta complessità.

4.1 Dataset Paper: MVTec AD

Il dataset utilizzato, per gli esperimenti, nel paper originale è **MVTec AD**. Questo è composto da **15 classi totali**, suddivise in 10 classi di "oggetti" e 5 classi di "texture". Rappresenta un eccellente benchmark iniziale per l'ispezione visiva industriale, tuttavia, le recenti architetture di Deep Learning riescono ormai a risolverlo con alti tassi di accuratezza.

4.2 Dataset VisA

Per valutare l'effettiva scalabilità e robustezza dei metodi sviluppati è stato utilizzato il VisA Dataset. Introdotto più recentemente, VisA con-

tiene un totale di 10.821 immagini ad alta risoluzione (9.621 immagini normali e 1.200 anomale), suddivise in **12 differenti prodotti**.

VisA introduce una serie di caratteristiche e sfide che lo rendono un banco di prova di altissimo livello per il Continual Anomaly Detection:

- **Complessità Strutturale:** A differenza di MVTec in cui l'oggetto è spesso un singolo elemento posto centralmente, VisA include oggetti con strutture complesse e molteplici sotto-componenti (es. PCB - circuiti stampati, capsule).
- **Posizionamento Multiplo:** Alcune classi presentano istanze multiple dello stesso oggetto all'interno della medesima immagine, disposte in posizioni o orientamenti casuali (es. maccheroni).
- **Difetti Logici e Sottili:** Oltre ai difetti di superficie classici (graffi, buchi), VisA contiene difetti logici e strutturali molto più sottili (es. un pin mancante in una scheda elettronica).

Perché VisA è una sfida per il CAD? Queste complessità mettono a dura prova lo spazio latente del modello. Discriminare i difetti sottili di un PCB richiede l'estrazione di feature molto specifiche, che sono estremamente fragili e facilmente soggette al catastrophic forgetting quando il modello viene successivamente addestrato a riconoscere una classe strutturalmente opposta (come ad esempio le texture o le capsule).

4.3 Definizione dei Task Sequenziali

Poiché l'obiettivo è addestrare il modello in modalità "Continual", le classi dei dataset non vengono processate contemporaneamente, pertanto, questi sono stati partizionati in "task" temporali.

Un **task** rappresenta un gruppo di nuovi prodotti che la linea di produzione deve imparare a ispezionare in un dato momento.

Sono stati implementati due scenari principali di addestramento incrementale:

- **Scenario Multi-category Incremental Learning (MCIL):** In questo assetto le classi del dataset vengono divise in blocchi uniformi (ad esempio N classi per ogni task). Ogni task sottopone al modello un gruppo completamente nuovo ed eterogeneo di prodotti, forzandolo a espandere drasticamente i propri orizzonti decisionali a ogni epoca incrementale.
- **Scenario Incremental con Inizializzazione Bulk:** In questo specifico scenario, il modello affronta un primo task massiccio contenente un blocco consistente di classi. Lo scopo è fornire alla rete neural un’ottima inizializzazione domain-aware. Nei task successivi (task $t \geq 1$), i restanti prodotti vengono presentati sequenzialmente, tipicamente una singola classe per volta. Questo simula uno scenario industriale iper-realistico in cui un’azienda addestra inizialmente il proprio macchinario ispettore su tutto il suo catalogo pre-esistente (le prime 8 classi), e nel corso dell’anno aggiunge dinamicamente nuovi articoli al catalogo, uno dopo l’altro.

Capitolo 5

Risultati Sperimentali

In questo capitolo vengono presentati e analizzati i risultati ottenuti dai diversi approcci di Continual Anomaly Detection. L'analisi è suddivisa in base ai due dataset di riferimento utilizzati per la sperimentazione: **MVTec AD** e **ViSA**. Per ciascun dataset vengono esposte le performance dei tre metodi proposti (DNE , $DNE + EWC$, $DNE + Replay + EWC$), valutandone l'efficacia al variare dell'architettura di estrazione delle feature (ViT , $EfficientNet$, $ConvNeXt$).

5.1 Risultati ottenuti su Dataset MVTEC AD

5.1.1 Analisi del metodo base: DNE

In una prima fase di test, è stata confrontata l'architettura originale basata su self-attention (ViT-16) contro le moderne architetture puramente convoluzionali (EfficientNet e ConvNeXt) utilizzando unicamente la baseline DNE standard.

Analisi delle Prestazioni Globali

Osservando le metriche complessive, emerge immediatamente una spaccatura tra il Transformer e le CNN. Il ViT-16 raggiunge un'ottima Ave-

rage Accuracy finale dell'81.62%, mantenendo un Forgetting Measure contenuto (7.28%). Al contrario, le reti convoluzionali mostrano performance nettamente inferiori: ConvNeXt si ferma al 65.86% di ACC, mentre EfficientNet crolla al 63.53% con un tasso di dimenticanza globale altissimo (19.81%). Questo evidenzia come, in assenza di meccanismi mitigativi, i Transformer gestiscano molto meglio i task incrementali rispetto alle classiche architetture CNN.

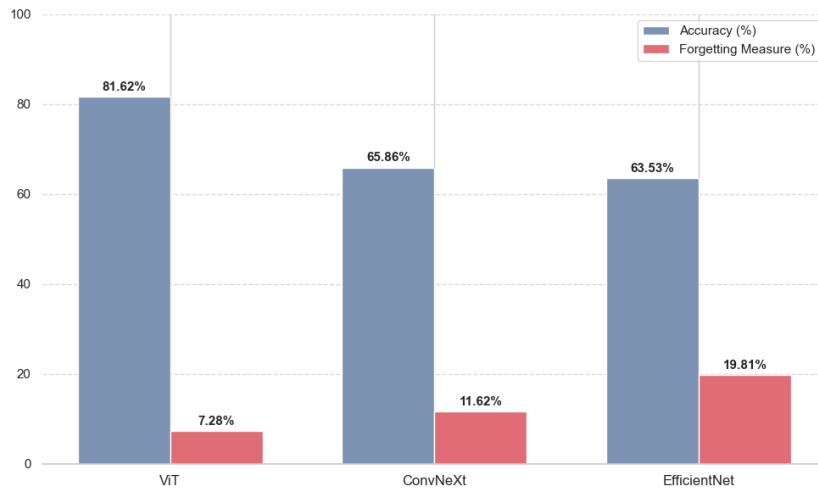


Figura 5.1: Confronto Finale: Mean AUC vs Forgetting Measure

Analisi Task-by-Task

Analizzando l'andamento sequenziale durante la fase di training, si osserva che le CNN (ConvNeXt, EfficientNet) faticano enormemente a ritenere la conoscenza pregressa ogni volta che un nuovo task viene presentato. Mentre il ViT-16 riesce a mantenere alte performance su quasi tutti i task sequenziali con alcuni cali dovuti alla complessità dei task.

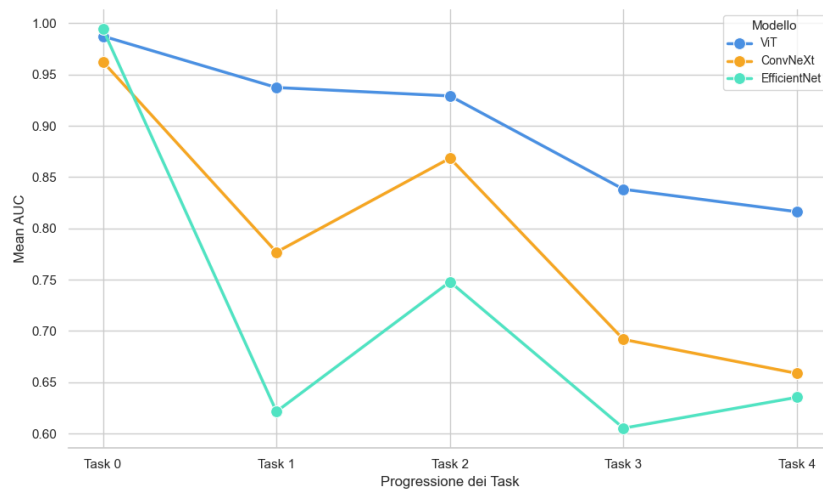


Figura 5.2: Evoluzione della Mean AUC per Task

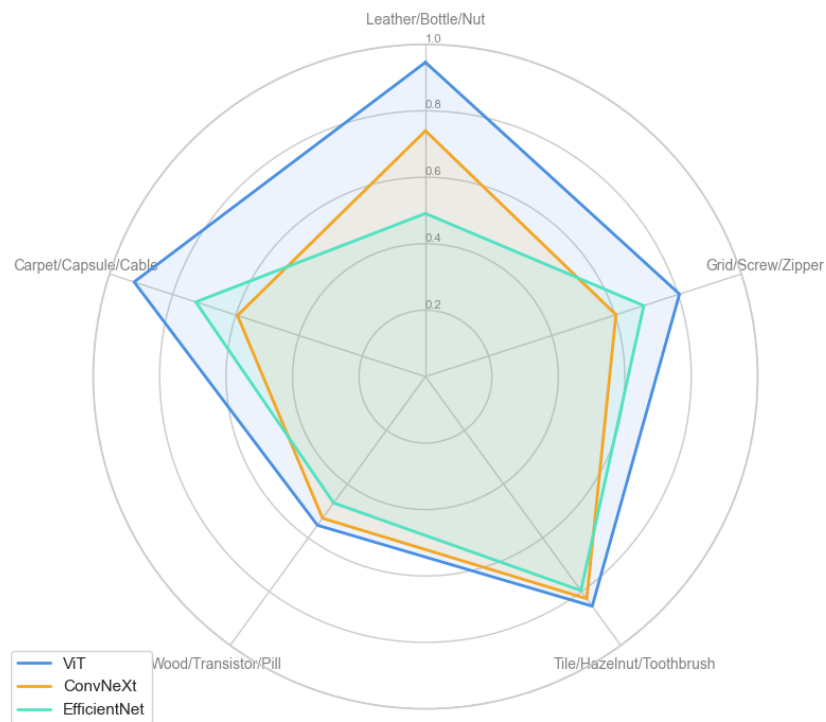


Figura 5.3: Confronto Final AUC sui Singoli Set di Dati



Figura 5.4: Evoluzione dell'AUC dei Task durante l'addestramento

5.1.2 Analisi del metodo DNE_EWC

Per mitigare il catastrophic forgetting, in particolare delle CNN, evidenziato nella baseline, è stata introdotta la regolarizzazione EWC, mirata a limitare l'alterazione dei pesi più importanti per i vecchi task.

Analisi delle Prestazioni Globali

L'introduzione della **matrice di Fisher** e della **penalizzazione EWC** produce un innalzamento globale delle prestazioni su tutta la linea. Il ViT-16 migliora sfiorando l'**84% di ACC** e dimezzando quasi il forgetting (**4.76%**). L'impatto più evidente si nota però sulle CNN: Effi-

cientNet e ConvNeXt salgono appaiate a circa il **69.5% di ACC**. Le **metriche di FM** calano sensibilmente, dimostrando che la costrizione elastica sui pesi funziona e argina parzialmente il decadimento delle prestazioni medie.

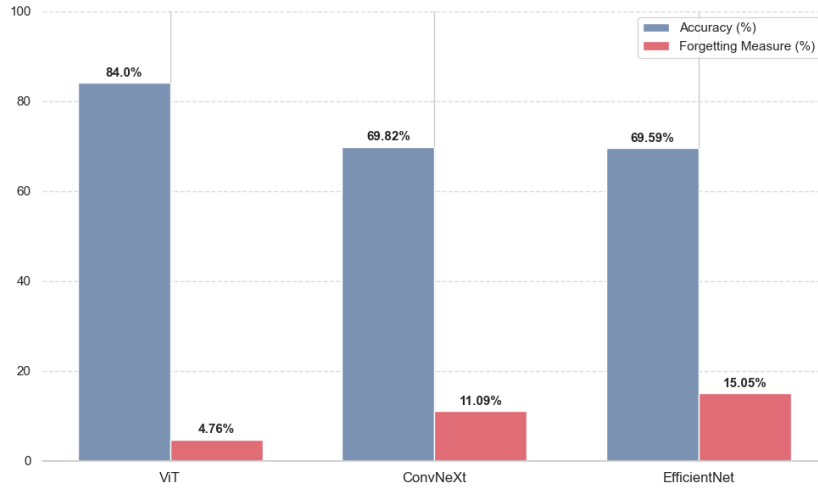


Figura 5.5: Confronto Finale: Mean AUC vs Forgetting Measure

Analisi Task-by-Task

L'analisi sequenziale task-by-task rivela che l'EWC costringe la rete a **preservare i pesi ottimali** per le anomalie e i prodotti già visti. Questo stabilizza notevolmente la memoria delle reti convoluzionali lungo il susseguirsi delle epoche, proteggendo vigorosamente l'accuratezza sui task appresi in passato. Tuttavia, la rigidità imposta al network ostacola in parte l'assimilazione ottimale dei **task intermedi**: la rete sacrifica parzialmente la capacità di modellare perfettamente i nuovi dati pur di non distruggere le feature precedenti. Questo compromesso si traduce in un **Forgetting finale molto più contenuto**.

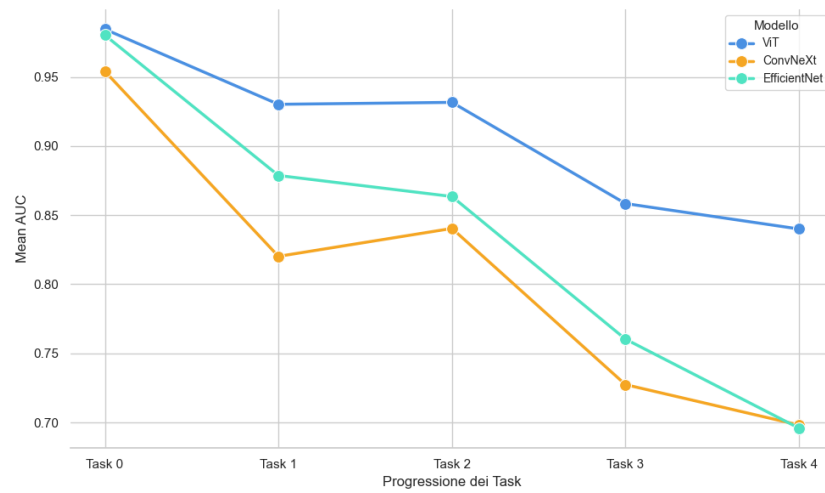


Figura 5.6: Evoluzione della Mean AUC per Task

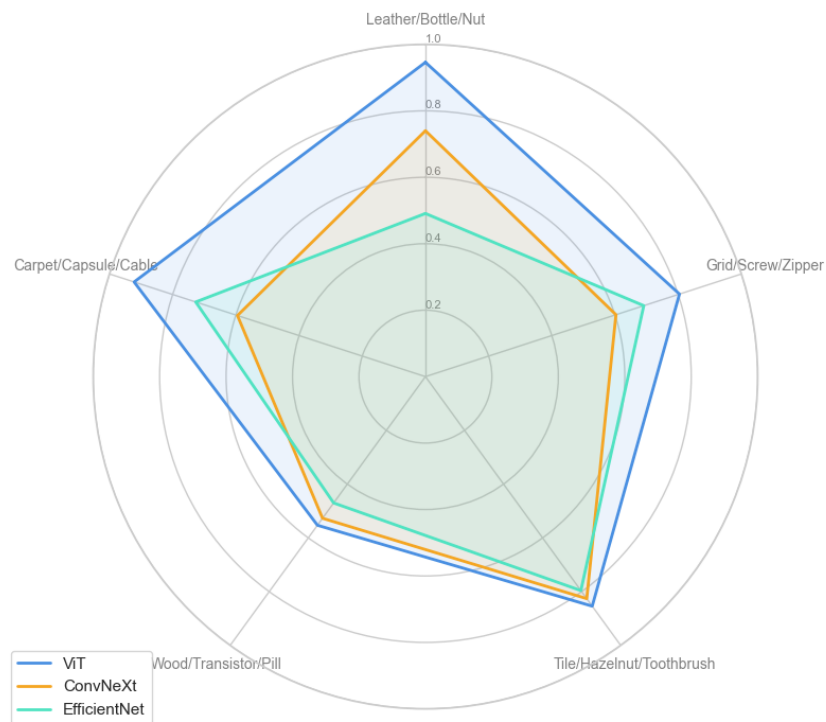


Figura 5.7: Confronto Final AUC sui Singoli Set di Dati



Figura 5.8: Evoluzione dell'AUC dei Task durante l'addestramento

5.1.3 Analisi del metodo DNE_replay_EWC

Il punto di svolta definitivo si evince analizzando il metodo ibrido DNE_Replay_EWC, progettato per unire la regolarizzazione (EWC) al ricalcolo on-the-fly delle vecchie feature tramite un buffer di memoria (**Image Replay**).

Analisi delle Prestazioni Globali

Le performance generali di questo approccio rappresentano i **risultati migliori**. Il ViT-16 raggiunge quasi il **90% di ACC**, azzerando virtualmente il **Forgetting Measure (0.75%)**. Le CNN subiscono una

metamorfosi: EfficientNet compie un salto enorme raggiungendo quasi il **79% di ACC** e portando la sua FM a un eccellente **4.99%** (valore diviso per 4 rispetto al DNE standard!). Anche ConvNeXt consolida la sua accuratezza al **74.6%** con un FM identico (**4.99%**). Questo dimostra che il **ricalcolo delle centroidi gaussiane** ha colmato la falla strutturale delle CNN nel Continual Learning.

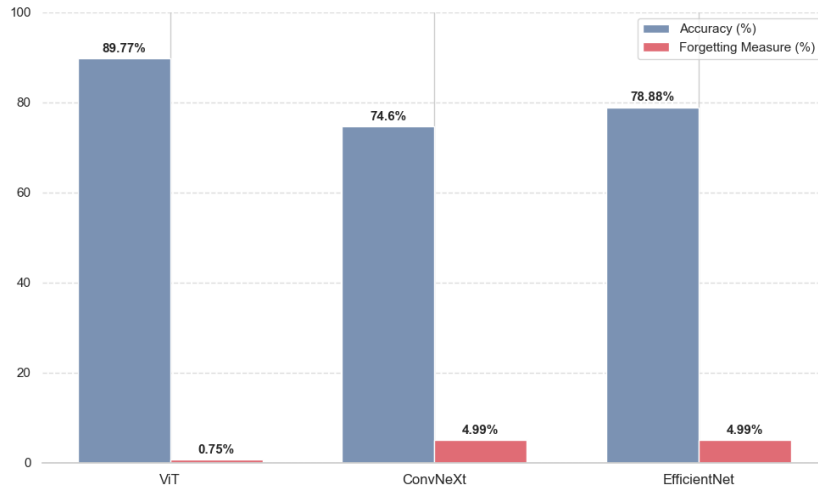


Figura 5.9: Confronto Finale: Mean AUC vs Forgetting Measure

Analisi Task-by-Task

Aggiungendo l'**Image Replay** per il ricalcolo continuo delle statistiche di densità, l'andamento task-by-task diventa **eccellente e stabile** su tutte le architetture. Durante l'acquisizione in sequenza dei vari prodotti, ogni modello aggiorna dinamicamente il proprio spazio latente, **prevenendo del tutto il disallineamento matematico**. Le CNN non mostrano più il drammatico effetto di sovrascrittura delle informazioni: le **distribuzioni gaussiane rimangono coerenti e centrate** per ogni task dall'inizio alla fine dell'esperimento.

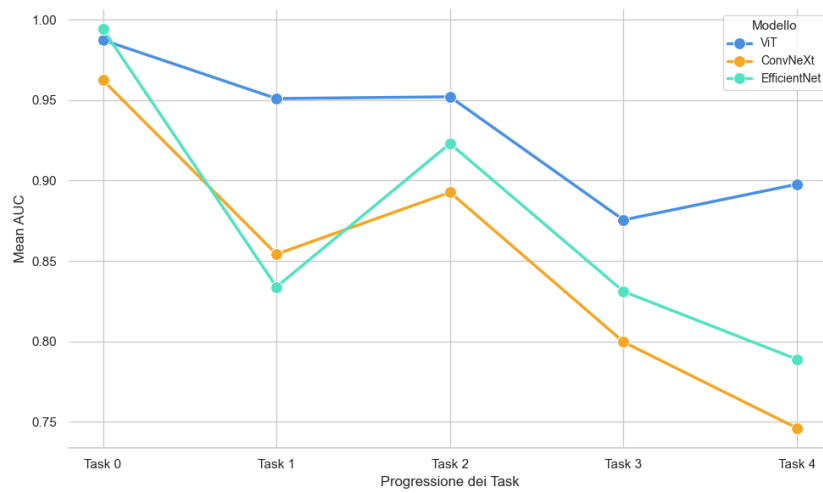


Figura 5.10: Evoluzione della Mean AUC per Task

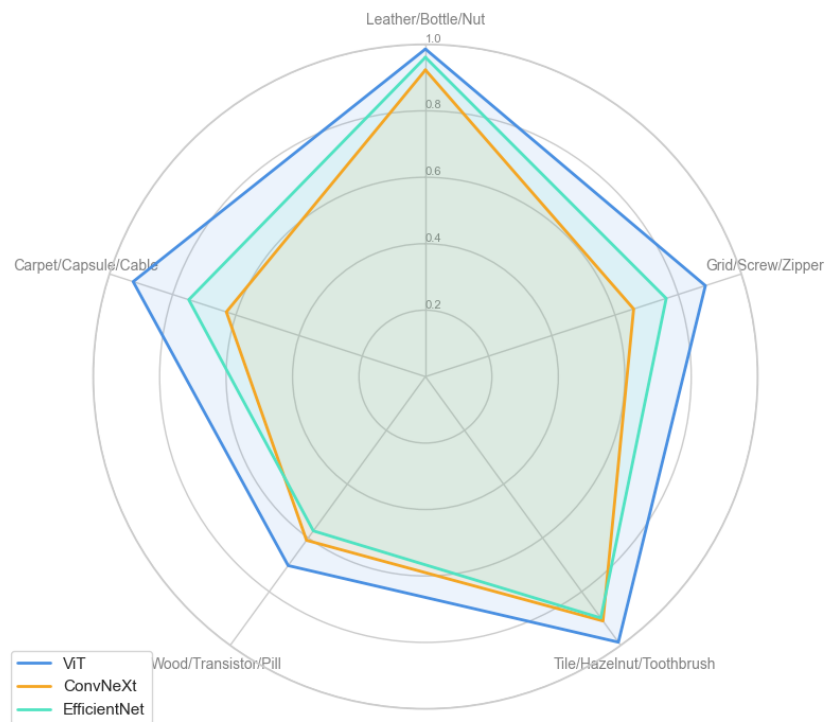


Figura 5.11: Confronto Final AUC sui Singoli Set di Dati

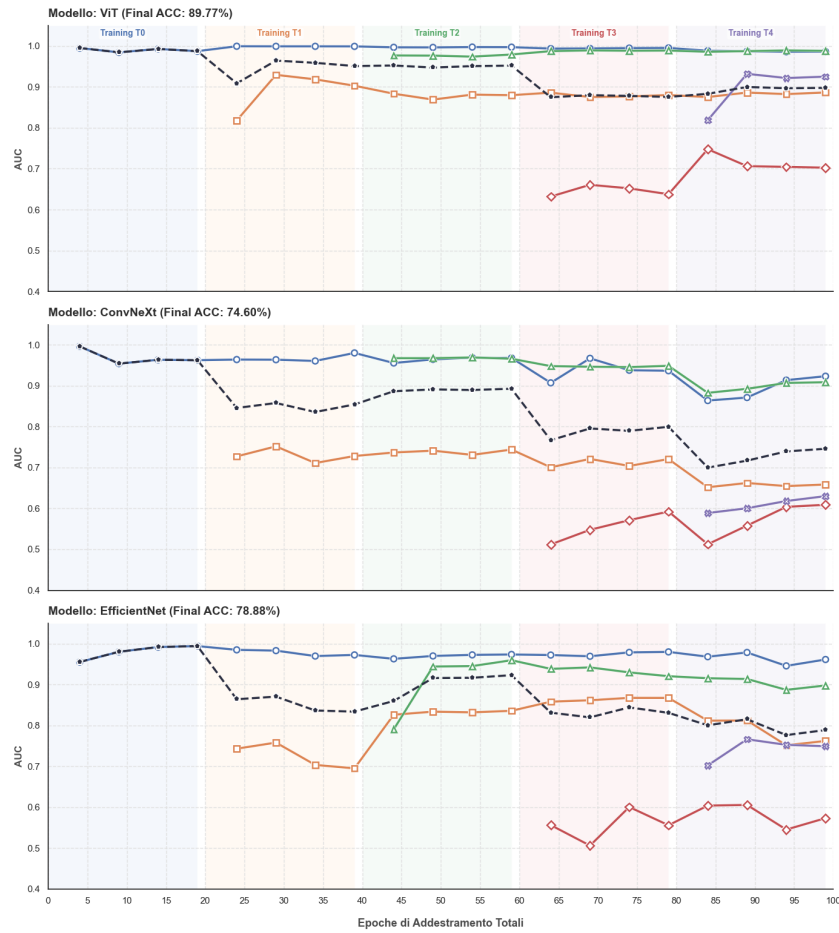


Figura 5.12: Evoluzione dell'AUC dei Task durante l'addestramento

Visualizzazione Qualitativa (t-SNE e Istogrammi)

A conferma degli eccellenti risultati quantitativi ottenuti dall'architettura ViT-16 equipaggiata con il metodo DNE_Replay_EWC, viene di seguito proposta un'analisi qualitativa dello spazio latente e delle distribuzioni dei punteggi di anomalia.

La visualizzazione tramite **t-SNE** permette di mappare in due dimensioni la topologia dello spazio delle feature appreso dal modello.

Parallelamente, l'**Istogramma dei Punteggi di Anomalia** mostra la distribuzione delle distanze di Mahalanobis calcolate durante la stima di densità (rappresentate sull'asse x come $-\log(p(z))$). La netta separa-

zione visiva tra la distribuzione dei campioni sani (in azzurro) e quella dei campioni difettosi (in rosso) certifica visivamente l’altissima capacità discriminativa raggiunta dal modello su MVTec.

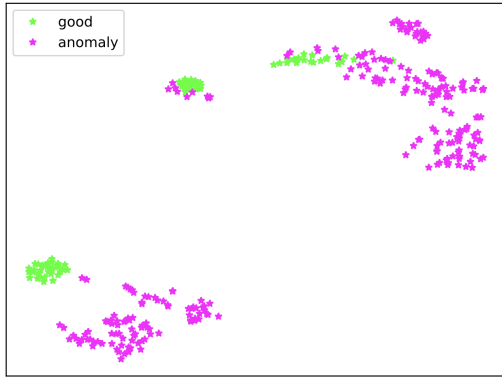


Figura 5.13: Spazio latente (t-SNE) per ViT-16 con metodo DNE_replay_EWC su MVTec su task 0 alla fine dell’addestramento.

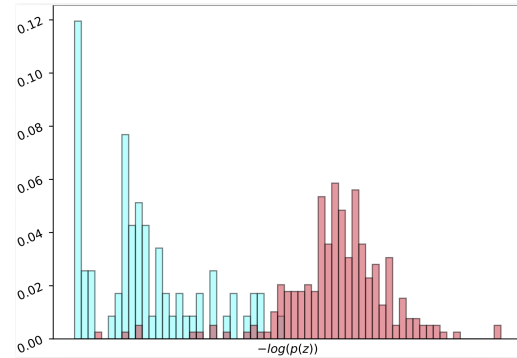


Figura 5.14: Distribuzione degli anomaly score per ViT-16 con metodo DNE_replay_EWC su MVTec su task 0 alla fine dell’addestramento..

5.1.4 Iperparametri di Addestramento

Per l’addestramento sul dataset MVTec (nella configurazione sequenziale `seq-mvtec`), il training è stato strutturato in **5 task incrementali**, ciascuno contenente 3 classi (`data_incre_setting: mul`), con l’applicazione di tecniche di **strong augmentation** su immagini ridimensionate a **224x224** pixel.

Iperparametri di Ottimizzazione

Tutti i metodi condividono i seguenti parametri di base:

- **Ottimizzatore:** Adam
- **Learning Rate:** 1×10^{-4}
- **Weight Decay:** 3×10^{-5}
- **Momentum:** 0.9

- **Batch Size:** 8
- **Epoche per Task:** 20

Metodologie e Parametri Specifici

Tutti i metodi testati si basano sul framework **DNE** e condividono di default i seguenti parametri:

- **Congelamento della Testa:** `fix_head: True`

Di seguito i parametri aggiuntivi per le singole varianti:

- **Metodo Base (DNE):**
Nessun parametro aggiuntivo.
- **Metodo DNE_EWC:**
* **EWC** $\lambda = 5000.0$
- **Metodo DNE-Replay_EWC:**
* **EWC** $\lambda = 5000.0$
* **Real Embed Ratio:** 0.05

Ambiente di Esecuzione

Gli esperimenti su questo dataset sono stati condotti su un MacBook Pro equipaggiato con processore **Apple Silicon M5**.

5.2 Risultati ottenuti su Dataset VisA

Per testare ulteriormente le architetture e i metodi proposti, gli esperimenti sono stati replicati sul dataset VisA. Questo dataset rappresenta una **sfida intrinsecamente più ardua** rispetto a MVTec AD.

L'analisi che segue traccia le performance su VisA per i modelli ViT-16 ed EfficientNet, evidenziando come la **maggiore complessità del dato** influisca sul catastrophic forgetting.

5.2.1 Analisi del metodo base: DNE

La baseline DNE è stata testata sulle immagini di VisA per quantificare la resilienza di base del Transformer rispetto alle reti convoluzionali su feature complesse.

Analisi delle Prestazioni Globali

Il passaggio al dataset VisA fa emergere prepotentemente le difficoltà intrinseche del Continual Anomaly Detection e quelle legate alla complessità dei task del dataset stesso. Il ViT-16 si ferma al **60.45% di Average Accuracy** complessiva, subendo un Forgetting Measure del **17.04%** (quasi triplicato rispetto ai risultati su MVTec). Le architetture convoluzionali risentono ancora di più della complessità dei difetti: EfficientNet crolla al **52.56% di ACC**, con un tasso di dimenticanza altissimo che sfiora il **22.5%**. La complessità strutturale dei difetti di VisA spinge la baseline DNE al **limite delle proprie capacità**.

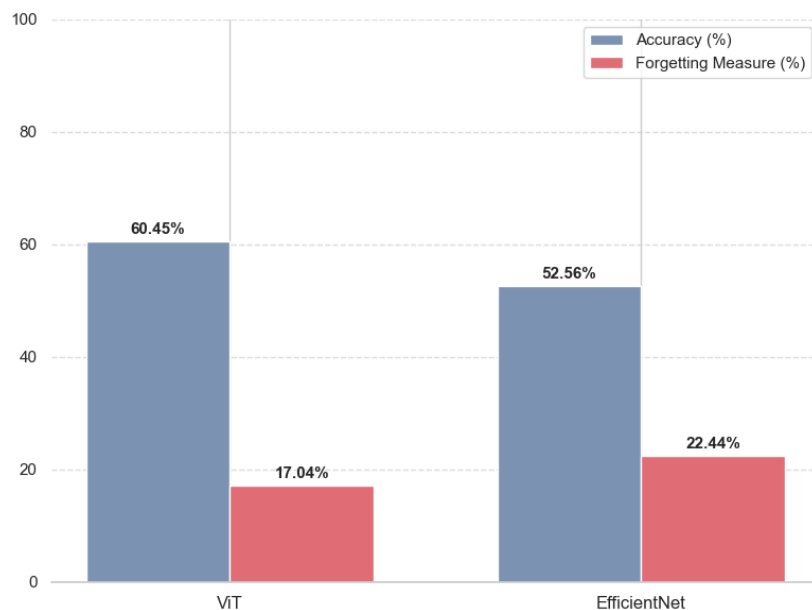


Figura 5.15: Confronto Finale: Mean AUC vs Forgetting Measure

Analisi Task-by-Task

L'andamento sequenziale mostra come la complessità degli sfondi di Visa causi un **rapido collasso delle performance** sui vecchi prodotti. Analizzando l'andamento sequenziale, si osserva che la complessità degli sfondi di Visa innesca un **grave collasso a cascata** lungo tutta la sequenza dei task. Quando i modelli tentano di assimilare nuove classi di difetti, le **distribuzioni gaussiane apprese sui prodotti precedenti** vengono **inesorabilmente distorte**.

Nella rete EfficientNet, questa **amnesia è drastica**: il decadimento non colpisce solo le primissime memorie, ma si abbatte violentemente anche sui **task intermedi**, degradando rapidamente le prestazioni a **livelli prossimi alla classificazione casuale** per quasi tutte le vecchie classi. Il ViT-16 resiste parzialmente, ma subisce comunque una **costante e diffusa erosione prestazionale** man mano che il training avanza.

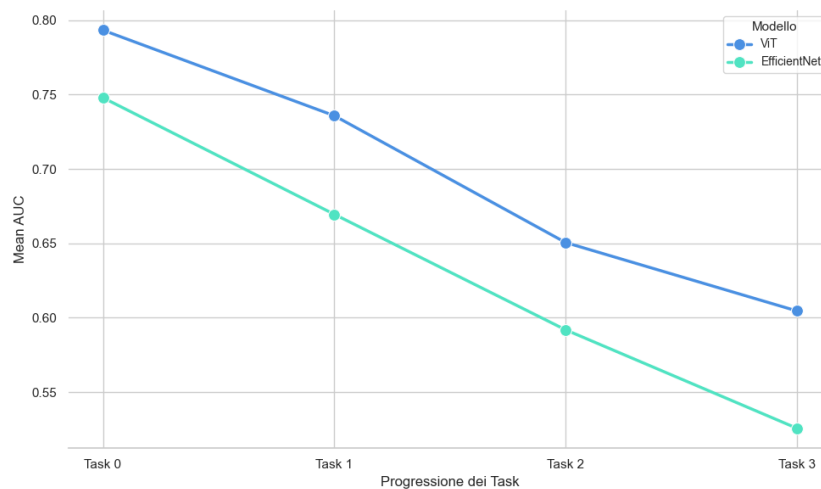


Figura 5.16: Evoluzione della Mean AUC per Task



Figura 5.17: Confronto Final AUC sui Singoli Set di Dati

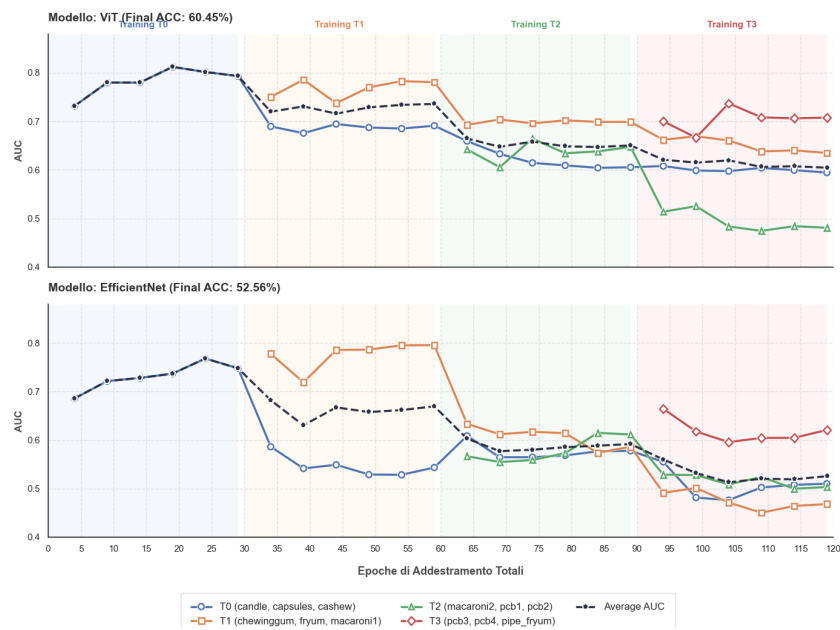


Figura 5.18: Evoluzione dell'AUC dei Task durante l'addestramento

5.2.2 Analisi del metodo DNE_EWC

Analisi delle Prestazioni Globali

L'applicazione di EWC su VisA offre risultati molto interessanti e contrastanti. Da un lato, EfficientNet beneficia in modo netto del vincolo: la sua ACC sale al **56.12%** e l'FM scende sotto il **17%**. Questo dimostra che arginare il **drastico stravolgimento dei pesi convoluzionali** è vitale per le CNN su dataset difficili. Dall'altro lato, il ViT-16 mostra un andamento sostanzialmente invariato (ACC 60.20%, FM 18.03%): il Transformer, avendo già una sua stabilità strutturale intrinseca, non trae vantaggi significativi dalla semplice limitazione dei pesi in contesti di alta eterogeneità di background.

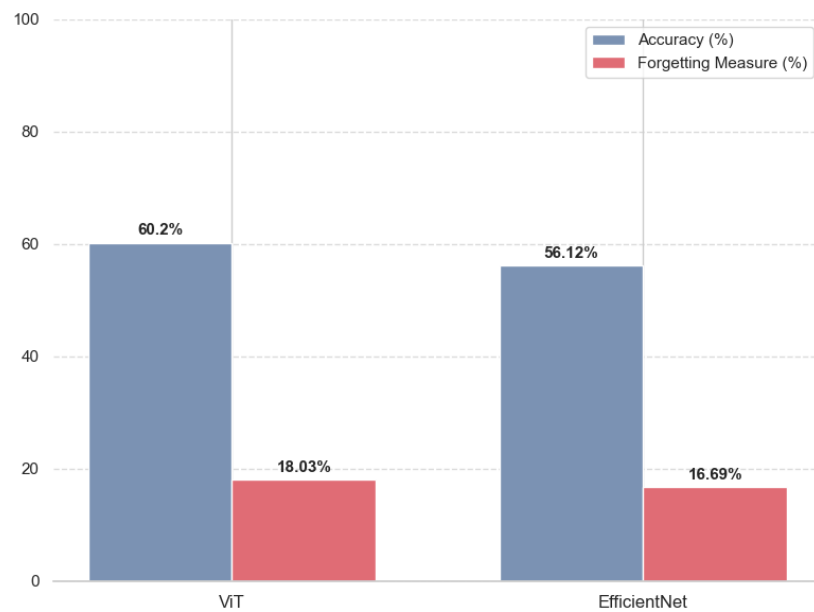


Figura 5.19: Confronto Finale: Mean AUC vs Forgetting Measure

Analisi Task-by-Task

La regolarizzazione dei pesi protegge in maniera eccellente le primissime classi apprese, **bloccando lo stravolgimento delle feature basilari** (un effetto salvifico particolarmente evidente su EfficientNet, che blin-

da la memoria iniziale). Tuttavia, la **rigidità imposta alla rete** per proteggere le vecchie memorie impedisce ad essa di riadattarsi correttamente alla sfaccettata eterogeneità di VisA nei task successivi. Di conseguenza, mentre la memoria del primissimo task viene salvaguardata, le categorie apprese nel mezzo della sequenza collassano ugualmente a causa della **mancanza di plasticità dei pesi convoluzionali**, limitando drasticamente i benefici sull'accuratezza media globale.

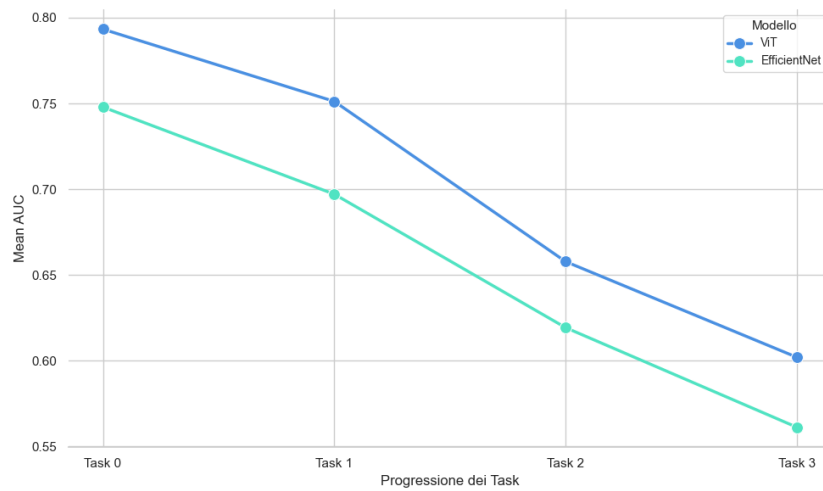


Figura 5.20: Evoluzione della Mean AUC per Task



Figura 5.21: Confronto Final AUC sui Singoli Set di Dati

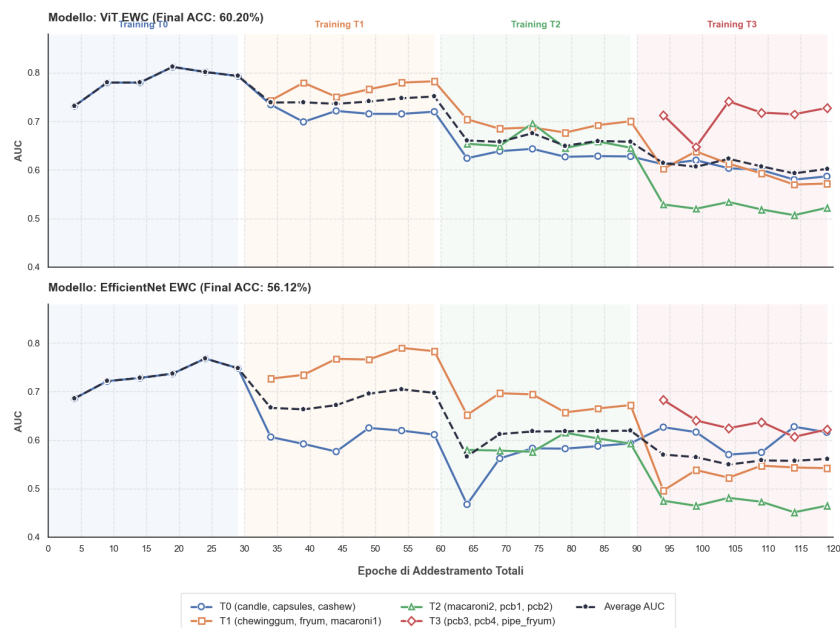


Figura 5.22: Evoluzione dell'AUC dei Task durante l'addestramento

5.2.3 Analisi del metodo DNE_replay_EWC

Analisi delle Prestazioni Globali

I risultati dell'approccio ibrido dimostrano che il framework sviluppato scala con successo anche su dataset ostici. Il ViT-16 ottiene le migliori prestazioni assolute dell'intero esperimento VisA: l'Average Accuracy compie un balzo notevole, raggiungendo il **67.40%**. Ma il dato più sbalorditivo è il Forgetting Measure, che crolla clamorosamente all'**8.58%** (praticamente dimezzato rispetto all'oltre 17% del DNE base e del DNE_EWC). Anche EfficientNet registra un riscatto enorme: l'Average Accuracy sale al **61.15%** (rispetto al 52.56% della baseline) e il Forgetting crolla al **13.01%** (dal 22.44% di partenza).

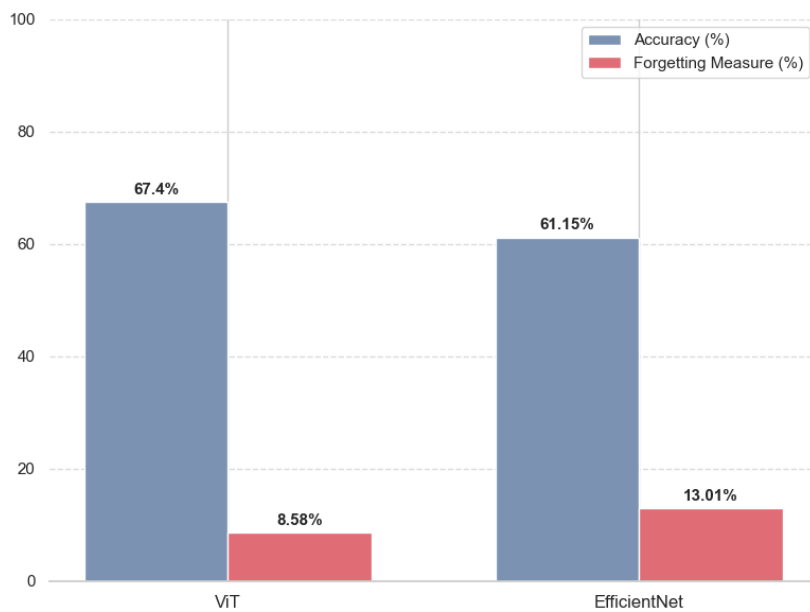


Figura 5.23: Confronto Finale: Mean AUC vs Forgetting Measure

Analisi Task-by-Task

L'analisi dinamica conferma che il ricalcolo continuo rappresenta la chiave di volta per stabilizzare l'andamento sequenziale. L'ibridazione tra Replay ed EWC **annulla quasi totalmente il disallineamento dello**

spazio latente lungo l'intera traiettoria di training. Il **decadimento a cascata viene inibito**: sia EfficientNet che il ViT-16 riescono ora a ritenere saldamente la conoscenza di tutta la sequenza di prodotti, bloccando la distorsione delle feature e consentendo alle architetture di performare in modo robusto anche sui complessi background di VisA.

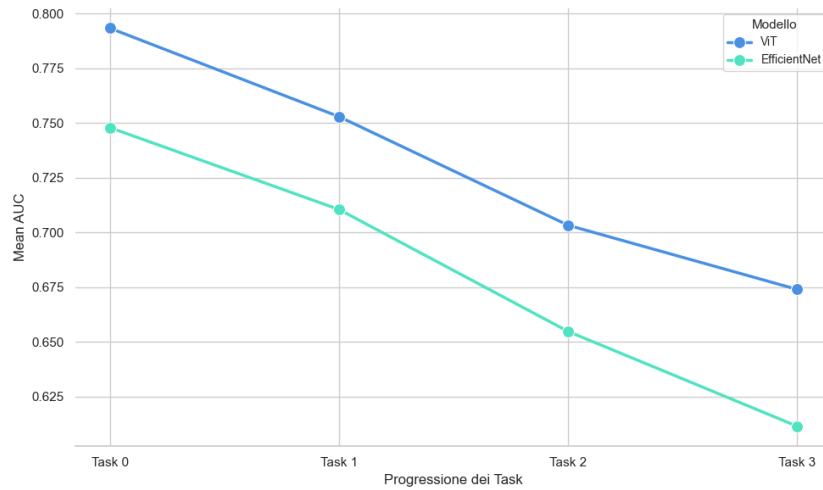


Figura 5.24: Evoluzione della Mean AUC per Task



Figura 5.25: Confronto Final AUC sui Singoli Set di Dati

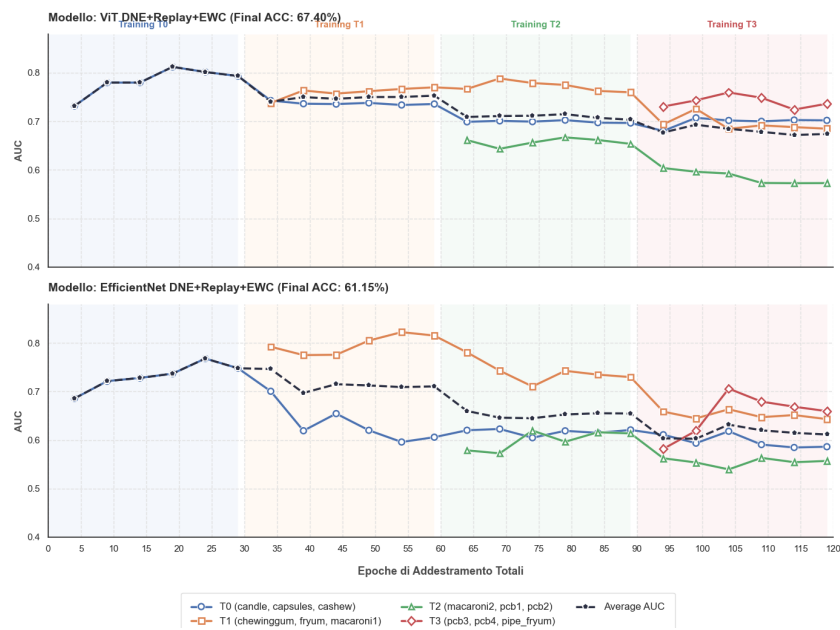


Figura 5.26: Evoluzione dell'AUC dei Task durante l'addestramento

Visualizzazione Qualitativa (t-SNE e Istogrammi)

Analogamente a quanto fatto per MVTec, si analizza visivamente il comportamento del miglior modello emerso su VisA (ViT-16 con DNE_Replay_EWC). Vista la maggiore eterogeneità e complessità dei background di questo dataset, la capacità del modello di organizzare lo spazio latente è sottoposta a uno stress test molto più severo.

I grafici mostrano che il modello sta preservando una certa separazione, ma sperimenta difficoltà a causa del catastrophic forgetting e alle difficoltà intrinseche del dataset.

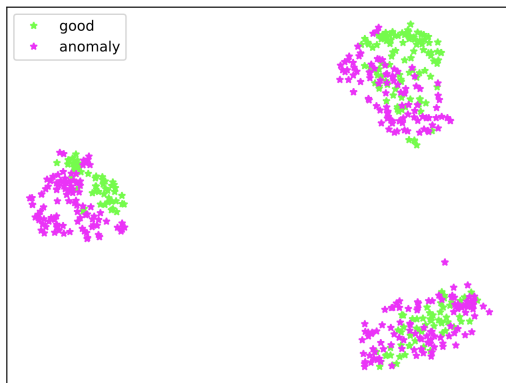


Figura 5.27: Spazio Latente (t-SNE) per ViT-16 con metodo DNE_replay_EWC su VisA su task 0 alla fine dell'addestramento.

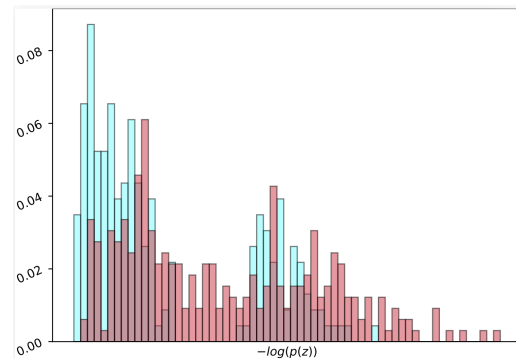


Figura 5.28: Distribuzione Anomaly Score per ViT-16 con metodo DNE_replay_EWC su VisA su task 0 alla fine dell'addestramento.

5.2.4 Iperparametri di Addestramento

Per l'addestramento sul dataset VisA (configurazione `seq-visa`), il training prevede **4 task incrementali** da 3 classi ciascuno (`data_incre_setting: mul`), con **strong augmentation** e immagini a **224x224** pixel.

Iperparametri di Ottimizzazione

Tutti i metodi condividono i seguenti parametri di base:

- **Ottimizzatore:** Adam
- **Learning Rate:** 1×10^{-4}
- **Weight Decay:** 3×10^{-5}
- **Momentum:** 0.9
- **Batch Size:** 32
- **Epoche per Task:** 30

Metodologie e Parametri Specifici

Tutti i metodi testati si basano sul framework **DNE** e condividono di default i seguenti parametri:

- **Congelamento della Testa:** `fix_head: True`

Di seguito i parametri aggiuntivi per le singole varianti:

- **Metodo Base (DNE):**
Nessun parametro aggiuntivo.
- **Metodo DNE-EWC:**
 - * **EWC** $\lambda = 5000.0$
- **Metodo DNE-Replay-EWC:**
 - * **EWC** $\lambda = 5000.0$
 - * **Real Embed Ratio:** 0.05

Ambiente di Esecuzione

Gli esperimenti su questo dataset sono stati condotti in ambiente Google Colab, sfruttando un'accelerazione hardware tramite **GPU NVIDIA A100**.

Capitolo 6

Conclusioni e Sviluppi Futuri

6.1 Conclusioni

Questo progetto ha affrontato il problema del *Catastrophic Forgetting* nel Continual Anomaly Detection (CAD) applicato all'ispezione visiva industriale. A partire dal framework DNE (Distribution of Normal Embeddings), sono state integrate architetture convoluzionali più efficienti, come EfficientNet e ConvNeXt, e sono state proposte due varianti algoritmiche, DNE_EWC e DNE_Replay_EWC, per migliorare la robustezza sui task già appresi.

L'analisi sperimentale ha evidenziato quattro risultati principali:

- **Limiti delle architetture convoluzionali.** Nel metodo DNE di base, EfficientNet e ConvNeXt risultano più sensibili all'apprendimento sequenziale rispetto a ViT-16.
- **Impatto della regolarizzazione EWC.** L'introduzione di Elastic Weight Consolidation stabilizza la memoria delle CNN, limitando la variazione dei pesi rilevanti per i task precedenti. Questo effetto è particolarmente evidente su VisA, ma introduce anche una maggiore rigidità nell'adattamento ai task intermedi.

- **Efficacia dell’approccio ibrido.** DNE_Replay_EWC, che combina EWC e Image Replay con il 5% dei campioni reali, è risultato il metodo più efficace. Su MVTec AD, ViT-16 raggiunge una Forgetting Measure quasi nulla (0.75%) e un’accuratezza prossima al 90%.
- **Robustezza su scenari complessi.** Sul dataset VisA, caratterizzato da difetti sottili e sfondi eterogenei, i metodi di base subiscono un forte degrado. Anche in questo scenario, DNE_Replay_EWC riduce sensibilmente il forgetting e migliora l’accuratezza media, soprattutto per ViT-16 ed EfficientNet.

6.2 Sviluppi Futuri

I risultati ottenuti suggeriscono alcune direzioni di approfondimento:

- **Selezione dei campioni di replay.** DNE_Replay_EWC è stato testato su una selezione casuale del 5% delle immagini. Strategie di *active sampling* potrebbero individuare campioni più rappresentativi, riducendo ulteriormente il costo di memoria.
- **Difetti logici e strutturali.** Su VisA l’accuratezza resta limitata dalla presenza di anomalie sottili e dipendenti dalla posizione. Un possibile sviluppo riguarda l’estrazione di feature locali più discriminative.
- **Tuning di EWC.** Il parametro EWC $\lambda = 5000.0$ impone un compromesso tra memoria, nel ricordare i task precedenti, e adattamento. Tecniche di calibrazione adattiva potrebbero preservare le feature consolidate senza ridurre eccessivamente la capacità di apprendere nuovi task.

Bibliografia

- [1] James Kirkpatrick et al. «Overcoming catastrophic forgetting in neural networks». In: Proceedings of the National Academy of Sciences 114.13 (mar. 2017), pp. 3521–3526. ISSN: 1091-6490. DOI: 10.1073/pnas.1611835114. URL: <http://dx.doi.org/10.1073/pnas.1611835114>.
- [2] Zhuang Liu et al. A ConvNet for the 2020s. 2022. arXiv: 2201.03545 [cs.CV]. URL: <https://arxiv.org/abs/2201.03545>.
- [3] Mingxing Tan e Quoc V. Le. EfficientNet: Rethinking Model Scaling for Convolutional 2020. arXiv: 1905.11946 [cs.LG]. URL: <https://arxiv.org/abs/1905.11946>.